ASMx Data Documentation

A documentation to the datasets associated with the *Calibrating Adaptive Smoothing Methods for Freeway Traffic Reconstruction* repository

What this document is. A user-oriented guide to every dataset in `data/` of the ASMx repository. For each file it explains contents, the meaning of columns or array axes, units, and how to load the data in Python. Dataset descriptions follow Section 3.1 of the accompanying paper *Calibrating Adaptive Smoothing Methods for Freeway Traffic Reconstruction*; the paper contains further methodological and experimental details. This guide aims to clarify the data at all stages (raw query records, processed raw data, preprocessed datasets, etc.).

Contents

1 Quick start

Just want to load the calibration data?

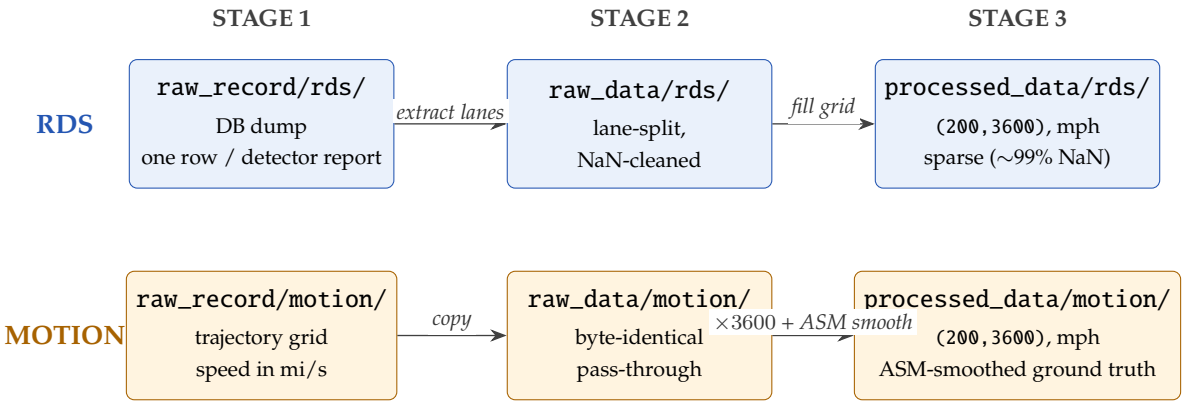
```python
import numpy as np

# Sparse RDS speed observations (the model input \tilde X in the paper)
rds = np.load('data/processed_data/rds/lane1/2024-07-09.npy')
# MOTION ground-truth speed field (the calibration target X-hat)
gt  = np.load('data/processed_data/motion/lane1/2024-07-09.npy')

print(rds.shape, gt.shape)   # (200, 3600) (200, 3600)
print(rds.dtype)             # float32
# Row index = space cell (200 cells from MM 58.7 to MM 62.7, dx = 0.02 mi)
# Col index = time cell (3600 cells from 06:00 to 10:00, dt = 4 s)
# Values    = speed in mph (multiply by 1.60934 for km/h, as the paper does)
```

The three-stage data pipeline at a glance

The two measurement modalities (RDS and MOTION) flow through the same three stages and meet on the same processed grid in Stage 3. The corridor-scale variant under `corridor_data/` mirrors Stage 3 for the wider 27.36 km window.



`corridor_data/` runs the same pipeline on a wider 27.36 km window → shape `(840, 3600)`.
Demo files at the root of `data/` are standalone (not produced by this pipeline).

Figure 1: The three-stage data pipeline. The RDS track produces the sparse model input $\tilde{\mathcal{X}}$; the MOTION track produces the dense ground truth $\hat{\mathcal{X}}$. Both end up on the same `(200,3600)` mph grid, so they can be overlaid without any rescaling.

What file do I need?

Stage	If you want to...	Open...
Stage 1 (`raw_record/`)	Inspect raw detector readings before any cleaning	`raw_record/rds/{date}.csv` (one row per detector report).
Stage 1 (`raw_record/`)	Re-run the preprocessing yourself end-to-end	Start from `raw_record/`, run scripts in `preprocessing/`.
Stage 2 (`raw_data/`)	Inspect lane-split, time-snapped detector readings	`raw_data/rds/{date}.csv` (one row per (MM, 30 s bin)).
Stage 3 (`processed_data/`)	Reproduce the lane-1 calibration in Sec. 3.2 (July 9, 2024)	`processed_data/rds/lane1/2024-07-09.npy` (model input) `processed_data/motion/lane1/2024-07-09.npy` (ground truth).
Stage 3 (`processed_data/`)	Run the cross-validation in Sec. 3.4	Both `processed_data/` trees, all five dates (`2024-07-08` ... `2024-07-12`), all four lanes.
Corridor (`corridor_data/`)	Reproduce the corridor-scale reconstruction in Fig. 13	`corridor_data/rds/lane1/2024-07-09.npy` (840×3600).
Demo (root of `data/`)	Reproduce the I-95 use case in Fig. 14	`I95speed.npy` (750×4088).

Repository layout at a glance

```
data/
|-- raw_record/        Stage 1: raw database dumps (one row per detector report)
|   |-- rds/{date}.csv
|   `-- motion/{date}/lane_{1..4}_speed_matrix.csv
|-- raw_data/          Stage 2: lane-split, time-snapped, NaN-cleaned CSVs
|   |-- rds/{date}.csv
|   `-- motion/{date}/lane_{1..4}_speed_matrix.csv
|-- processed_data/    Stage 3: regular (space, time) matrices, float32 .npy
|   |-- rds/lane{1..4}/{date}.npy        -> shape (200, 3600), sparse
|   `-- motion/lane{1..4}/{date}.npy     -> shape (200, 3600), dense ground truth
|-- corridor_data/     Same pipeline, but for the full 27.36 km SMART Corridor
|   `-- rds/{date}.csv  +  rds/lane{1..4}/{date}.npy  (shape (840, 3600))
|-- 2023-10-26.csv     Demo files, see Section 7
|-- 2025-04-09.csv
|-- I95speed.npy
|-- speed.npy
|-- asm_arxiv.pdf      The companion paper
`-- README.tex/.pdf    This document
```

2 What is in this dataset?

Site. I-24 westbound near Nashville, TN. The primary study area is the I-24 MOTION testbed (6.76 km of instrumented roadway, MM 58.7–62.7); demonstration files also cover the wider I-24 SMART Corridor (27.36 km, MM 53.3–70.1) and an I-95 northbound segment in Virginia (MM 115–130).

When. Five consecutive weekdays, 2024-07-08 through 2024-07-12 (Mon through Fri), 06:00–10:00 local time (America/Chicago). The canonical date list is `../dates.csv`.

Two measurement modalities are propagated through three pipeline stages:

- **RDS** (Radar Detector System). Fixed-location Wavetronix SmartSensor HD radar sensors at ≈483 m (≈0.3 mi) spacing, reporting 30-second aggregated speed, occupancy, and volume. This is the *sparse model input* $\tilde{\mathcal{X}}$ in the paper.
- **MOTION**. Vehicle-trajectory–derived speed field from the I-24 MOTION testbed, computed per Edie's definition at 4-second temporal and 32-meter (≈0.02 mi) spatial resolution using the parallelogram method. This is the *dense ground truth* $\hat{\mathcal{X}}$.

Tip

The two modalities are co-registered onto the same processed grid (shape `(200, 3600)`, $\Delta x = 0.02$ mi, $\Delta t = 4$ s, units of mph). You can overlay them directly without any rescaling.

2.1 Conventions used everywhere

Item	Value
Spatial domain (calibration)	Mile markers 58.7 → 62.7 (4 miles, westbound)
Spatial domain (corridor demo)	Mile markers 53.3 → 70.1 (I-24 SMART Corridor)
Time zone	America/Chicago (UTC−05:00 in summer)
Time-of-day window	06:00–10:00 local
Dates	2024-07-08, -09, -10, -11, -12 (Mon–Fri)
Lane numbering	`lane1` = leftmost (HOV/passing), `lane4` = rightmost. The two demo files in §7 use `lane0` … `lane3` — see the note there.
Time encoding	Unix epoch seconds. `time_unix_fix` is snapped to a 30-s grid by `round_to_fix` in `preprocessing/FAST.py`.
Missing values	Negative sensor readings → `NaN` in stage 2; unobserved cells in stage 3 stay `NaN`.
Processed-array axis order	`(space, time)` — row = mile-marker bin, column = time bin.
Processed-array dtype	`float32`.
Resolution of processed arrays	$\Delta x = 0.02$ mi (≈32 m), $\Delta t = 4$ s.

Heads-up

Speed unit on disk is mph. The paper figures present speeds in km/h. The conversion (`MILE_TO_KM = 1.60934`) is applied inside the visualization notebooks under `visualization/`. If your reproduction shows speeds off by a factor of ≈1.6, this is the reason.

2.2 The three-stage pipeline

```
raw_record/   ->   raw_data/   ->   processed_data/
 (DB dump)         (lane-split,      (regular space-time
                   NaN-cleaned)      matrices, .npy)
```

The same shape is used for both RDS and MOTION modalities. The corridor-scale variant (`corridor_data/`) follows the same stages but covers MM 53.3–70.1; it is used for the I-24 SMART Corridor demonstration (Fig. 13 in the paper).

3 Stage 1: raw_record/ — raw database dumps

Use this when you want the original, uncleaned readings. Exact dump of the upstream queries; one row per detector report; no cleaning applied.

3.1 raw_record/rds/{YYYY-MM-DD}.csv

What's in it. The full TDOT roadside-detector-system query for one day on I-24 westbound. Eight columns, one row per detector report (≈46 000 rows/day, ≈12 MB/day). Five files (one per date).

Header row.

```
link_update_time,speed,occupancy,volume,smooth_speed,smooth_occupancy,milemarker,lane_dict_text
```

Table 1: Columns of `raw_record/rds/{date}.csv`.

Column	Type	Unit	Meaning
`link_update_time`	ISO 8601, with TZ offset	—	Sensor report timestamp. Example: `2024-07-08 10:59:55.276203-05:00`.
`speed`	float	mph	Cross-lane mean speed at this mile marker, for this report.
`occupancy`	float	percent	Cross-lane mean lane occupancy.
`volume`	float	veh / report	Cross-lane aggregated vehicle count for this report.
`smooth_speed`	float	mph	Vendor-side smoothed speed (Wavetronix internal smoothing).
`smooth_occupancy`	float	percent	Vendor-side smoothed occupancy.
`milemarker`	float	miles	TDOT detector mile marker (location on I-24 westbound).
`lane_dict_text`	stringified Python dict	mixed	Per-lane raw values, of the form `{sensor_id: [name, speed_mph, volume, occupancy_pct], ...}`. Parsed by the `extract_lane_*` helpers in `preprocessing/FAST.py`. The lane index lives inside `name`, e.g. `R3G-00I24-59.0W (277)-Lane4`.

3.2 raw_record/motion/{YYYY-MM-DD}/lane_{1..4}_speed_matrix.csv

What's in it. The MOTION ground-truth speed field for one day, one lane, on a fixed space-time grid. Five dates × four lanes = **20 files**.

Layout.

- 3600 data rows × 215 columns, plus one header row (`0,1,...,214`) that just lists column indices — not data.
- Each row = one time step; each column = one space cell of the source MOTION grid. Axis coordinates are implicit (no embedded mile-marker or timestamp axis); they are reconstructed from the corridor grid in code.

Values. Speed in miles per second. Typical range ≈0.001–0.030 (= 3.6–108 mph). To convert to mph, multiply by 3600. `NaN` denotes a cell with no trajectory observation.

Tip

Stage 3 (§5.2) already handles the mi/s→mph conversion and the ASM smoothing for you, so unless you have a reason to re-process from scratch, just use `processed_data/motion/....npy`.

4 Stage 2: `raw_data/` — lane-split, cleaned CSVs

Use this when you want a per-(milemarker, 30-s bin) record but still want CSV rather than a matrix.

4.1 `raw_data/rds/{YYYY-MM-DD}.csv`

What's in it. Produced by `preprocessing/rds_fast_processing.py` (helpers in `preprocessing/FAST.py`). Each row is one (mile marker, 30 s time bin) cell, with per-lane speed/volume/occupancy broken out into separate columns. Five files.

Header row.

```
tdot_milemarker,time_unix_fix,lane1_speed,lane1_volume,lane1_occ,
lane2_speed,lane2_volume,lane2_occ,lane3_speed,lane3_volume,lane3_occ,
lane4_speed,lane4_volume,lane4_occ,milemarker
```

Table 2: Columns of `raw_data/rds/{date}.csv`.

Column	Type	Unit	Meaning
`tdot_milemarker`	float	miles	Original TDOT mile marker (pre-snap).
`time_unix_fix`	float	Unix seconds	Time snapped to a 30-s grid via `round_to_fix`.
`lane1_speed`	float	mph	Lane-1 (leftmost / HOV) speed.
`lane1_volume`	float	veh / 30-s bin	Lane-1 vehicle count over the 30-s bin.
`lane1_occ`	float	percent	Lane-1 occupancy.
`lane2_speed`	float	mph	Lane-2 speed.
`lane2_volume`	float	veh / 30-s bin	Lane-2 vehicle count over the 30-s bin.
`lane2_occ`	float	percent	Lane-2 occupancy.
`lane3_speed`	float	mph	Lane-3 speed.
`lane3_volume`	float	veh / 30-s bin	Lane-3 vehicle count over the 30-s bin.
`lane3_occ`	float	percent	Lane-3 occupancy.
`lane4_speed`	float	mph	Lane-4 (rightmost) speed.
`lane4_volume`	float	veh / 30-s bin	Lane-4 vehicle count over the 30-s bin.
`lane4_occ`	float	percent	Lane-4 occupancy.
`milemarker`	float	miles	Snapped mile marker, after the TDOT → closest-mile-marker remap defined at the top of `preprocessing/rds_fast_processing.py`.

Cleaning applied at this stage.

- Negative `speed`/`volume`/`occupancy` values become `NaN` (negatives are TDOT transmission errors).
- When some lanes report and others do not, the missing per-lane values are filled with the across-lane mean for that (mile marker, time) cell — see `raw_lane_level_df_process` in `preprocessing/FAST.py`.

4.2 `raw_data/motion/{YYYY-MM-DD}/lane_{1..4}_speed_matrix.csv`

Byte-identical pass-through of `raw_record/motion/.../lane_{1..4}_speed_matrix.csv`. The MOTION source already arrives clean, so no cleaning is applied. Same layout and units as §3.2. The file

exists so that MOTION has the same `raw_record` → `raw_data` → `processed_data` shape as RDS — it is not a different file.

5 Stage 3: processed_data/ — ready-to-load matrices

This is what calibration and evaluation actually load. Regular space-time matrices, `dtype=float32`, axis order `(space, time)`, units of **mph**.

5.1 processed_data/rds/lane{1..4}/{YYYY-MM-DD}.npy

5 dates × 4 lanes = **20 files**.

- **Producer.** preprocessing/preprocessing.py (`fill_space_time_matrix`).
- **Source.** raw_data/rds/{date}.csv.
- **Spatial grid.** `np.arange(58.7, 62.7, 0.02)` miles → **200 cells**. Matches the calibration window in Fig. 1 of the paper (MM 58.7–62.7).
- **Temporal grid.** 4-second step in Unix seconds, spanning each file's `time_unix_fix` range → **3600 cells** for the 4-hour window.
- **Shape.** `(200, 3600)`.
- **Values.** Speed in mph, ≈0–115.
- **Sparsity.** Cells with no detector report stay `NaN` — on a representative file ≈99.3% of cells are `NaN`, reflecting the sparseness of RDS coverage (this is exactly the sparse panel in Fig. 1a of the paper).
- **Used by.** Loaded as the sparse-observation tensor $\tilde{\mathcal{X}}$ (variable `sp_np`) in `calibration/train.py`, `calibration/train_day_to_day.py`, `calibration/seeds.py`.

5.2 processed_data/motion/lane{1..4}/{YYYY-MM-DD}.npy

20 files.

- **Producer.** production/ASMxMOTION.py (`main`).
- **Source.** raw_data/motion/{date}/lane_{lane}_speed_matrix.csv, sliced to `[:3600, :200]` and transposed.
- **Two transforms applied before saving:**
  1. **Unit conversion** to mph: `speed = 3600 * data` (production/ASMxMOTION.py, line 179). The upstream CSVs are in mi/s; the on-disk `.npy` is in mph.
  2. **Adaptive smoothing**: an `AdaptiveSmoothing` torch module (defined in the same file) is applied to densify and de-noise the field before save. The output is therefore *not* a raw downsample of the source — it is the ASM-smoothed ground truth used by calibration.
- **Shape.** `(200, 3600)`, `dtype=float32`.
- **Values.** Speed in mph, ≈0.2–101.5. Because of the smoothing step, there are typically 0 `NaN`s in the on-disk array.
- **Used by.** Loaded as the ground-truth tensor $\hat{\mathcal{X}}$ (variable `gt_np`) by `calibration/train.py` (lines 256, 271), `calibration/train_day_to_day.py`, `calibration/seeds.py`.

Tip

RDS-processed and MOTION-processed arrays live on the **same** `(200, 3600)` grid with the **same** mph unit. Overlay them with `plt.imshow(rds)` / `plt.imshow(gt)` side-by-side — no rescaling needed.

6 Corridor-scale data: `corridor_data/`

Use this for the I-24 SMART Corridor reconstruction shown in Section 4.3.1 / Figure 13 of the paper (27.36 km, MM 53.3–70.1).

6.1 `corridor_data/rds/{YYYY-MM-DD}.csv`

Same schema as `raw_data/rds/` (Table 2). The only difference is the mile-marker range: it covers the full I-24 SMART Corridor instead of the 58.7–62.7 calibration window. Five files, ≈22 800 rows/day. Produced by `preprocessing/corridor_fast_processing.py`.

6.2 `corridor_data/rds/lane{1..4}/{YYYY-MM-DD}.npy`

Lane-split processed matrices for the full corridor, produced by `preprocessing/corridor_preprocessing.py`. Same dtype/units as §5.1, but on the corridor-wide MM 53.3–70.1 grid (840 cells at $\Delta x = 0.02$ mi) — so the on-disk shape is `(840, 3600)`, not `(200, 3600)`. Loaded by `production/ASMxCorridor.py`.

7 Demonstration files at the root of `data/`

These ship alongside the pipeline files but are *demonstration assets*, not products of the documented pipeline.

Table 3: Demonstration files.

File	Shape / format	Unit	Notes
`2023-10-26.csv`	CSV: `tdot_milemarker`, `time_unix_fix`, `lane1_speed`... `lane4_occ`, `milemarker`	mph	Same schema as `raw_data/rds/...csv` (§4.1). Single-day demo from a date outside the 2024-07 study window.
`2025-04-09.csv`	CSV: `time_unix_fix`, `lane0_speed`, `lane1_speed`, `lane2_speed`, `lane3_speed`, `milemarker`, `time_unix`	mph	**Uses `lane0`... `lane3` numbering, not `lane1`... `lane4`.** Demo file used for the I-95 Virginia corridor case study (manuscript Sec. 4.3.2, MM 115–130, 11:05–15:35).
`I95speed.npy`	`(750, 4088)`, `float32`	mph (≈3–100)	I-95 northbound demo matrix, MM 115–130; reproduces Fig. 14 of the paper. Not produced by the documented pipeline.
`speed.npy`	`(150, 545)`, `float32`	mph (≈3–100)	Small demo speed matrix referenced by notebooks. Not produced by the documented pipeline.

For `2025-04-09.csv` specifically, the per-column meanings are: `time_unix_fix` (snapped Unix seconds), `lane0_speed`... `lane3_speed` (per-lane speeds in mph; lane 0 = leftmost), `milemarker` (VDOT mile marker on I-95 NB, $\in [115, 130]$), `time_unix` (raw Unix seconds before snapping).

8 Demos

Load all five days for one lane

```
import numpy as np, pandas as pd
dates = pd.read_csv('dates.csv')['date'].tolist()
rds_stack = np.stack([
    np.load(f'data/processed_data/rds/lane1/{d}.npy') for d in dates
])  # shape (5, 200, 3600)
```

Convert on-disk speeds (mph) to km/h to match the paper

```
MILE_TO_KM = 1.60934
gt_kmh = np.load('data/processed_data/motion/lane1/2024-07-09.npy') * MILE_TO_KM
```

Recover the mile-marker / time axes for a processed matrix

```
import numpy as np, pandas as pd
mm = np.arange(58.7, 62.7, 0.02)              # 200 mile markers
df = pd.read_csv('data/raw_data/rds/2024-07-09.csv')
t0, t1 = df['time_unix_fix'].min(), df['time_unix_fix'].max()
t  = np.arange(t0, t0 + 4*3600, 4)            # 3600 timestamps, 4 s step
print(mm.shape, t.shape)                      # (200,) (3600,)
```

Check the sparsity of an RDS observation

```
a = np.load('data/processed_data/rds/lane1/2024-07-09.npy')
print(f'NaN fraction: {np.isnan(a).mean():.4f}')   # ~0.993 on a typical file
```

Plot ground truth and sparse observation side by side

```
import matplotlib.pyplot as plt, numpy as np
gt  = np.load('data/processed_data/motion/lane1/2024-07-09.npy')
rds = np.load('data/processed_data/rds/lane1/2024-07-09.npy')
fig, ax = plt.subplots(1, 2, figsize=(12, 4), sharex=True, sharey=True)
ax[0].imshow(rds, aspect='auto', cmap='RdYlGn', vmin=0, vmax=70); ax[0].set_title('RDS (sparse)')
ax[1].imshow(gt,  aspect='auto', cmap='RdYlGn', vmin=0, vmax=70); ax[1].set_title('MOTION (ground truth)')
plt.show()
```

9 FAQ and known caveats

Why do my reconstructed speeds look 1.6× off? On-disk arrays are in mph; paper figures are in km/h. Multiply by `1.60934`. See the warning box in Section 2.

Why are most cells in `processed_data/rds/` NaN? RDS is sparse by construction — a detector only reports when something passes a fixed location. ≈99.3% `NaN` on a representative file is expected. This sparsity is exactly the problem ASM is designed to solve.

Are the demo `lane0`–`lane3` files compatible with the pipeline? No. The pipeline assumes `lane1`–`lane4`. The demo file `2025-04-09.csv` uses `lane0`–`lane3` because it comes from a different ingestion path (the Virginia I-95 corridor); treat it as a standalone artifact.

Where does Lane 1 point? `lane1` is the leftmost (HOV/passing) lane on I-24 westbound; `lane4` is the rightmost (adjacent to on/off ramps). This is the convention used in Figures 4 and 5 of the paper.

Why do some `lane_dict_text` entries mention `Lane5`? The extractors in `preprocessing/FAST.py` only consume `Lane1`–`Lane4`; any `Lane5` entries are ignored. This does not affect the shipped `processed_data/`.

Why does `processed_data/rds/` use MM 58.7–62.7 but `corridor_data/` use MM 53.3–70.1? The former is the calibration window (the MOTION testbed, used for Figs. 1, 4, 5); the latter is the demonstration corridor (used for Fig. 13). They are both intentional — pick the one that matches the experiment you are reproducing.

Questions or corrections? Please open an issue at `https://github.com/trafficwaves/ASMx`.